

PARCOURS-AI

Architecture Complète — Frontend (Next.js) & Backend
(FastAPI)

Septembre 2026 — Cotonou, Bénin

Structure de dossiers, stack, flux d'état, comportements UI, schéma JSON, backend FastAPI/Supabase
et intégration Gemini

SOMMAIRE

1. Stack Technique	3
2. Structure complète des dossiers	4
3. Flux de données et State Management	5
4. Comportement UI par interaction	6
5. Schéma JSON strict (contrat backend)	8
6. Spécifications de Performance (Frontend)	9
7. Architecture Backend — Structure FastAPI	10
8. Modèle de données Supabase	11
9. Fiches métiers béninoises (données de référence)	12
10. Interaction avec l'API Gemini	13
11. Sécurité et robustesse Backend	14
12. Code des endpoints FastAPI	15

1. STACK TECHNIQUE

La stack est structurée pour la rapidité de développement, la performance mobile et la robustesse du contrat de données entre le moteur IA et le frontend.

DOMAINE	TECHNOLOGIES	RÔLE
Framework	Next.js 15 (App Router), TypeScript	Routage, Server Components pour le contenu statique (page d'accueil), Client Components pour le chat interactif.
UI	Tailwind CSS, Framer Motion, shadcn/ui	Tailwind gère le style, Framer Motion gère les transitions (chat → dashboard), shadcn/ui fournit les composants accessibles (Accordion pour les CareerCard).
State management	Zustand (+ persist middleware)	Mémoire l'historique du chat et le résultat final. Le middleware <code>persist</code> sauvegarde l'état dans le localStorage, assurant la continuité de session en cas d'interruption réseau.
Requêtes API	TanStack Query (React Query)	Gère les états <code>loading/error/success</code> des appels au backend, avec retry automatique en cas de coupure réseau.
Validation	Zod	Valide côté client le JSON reçu du backend avant son intégration dans le state.
Backend	FastAPI (Python) + Supabase (Postgres)	FastAPI sécurise la clé API Gemini et orchestre le prompt. Supabase stocke les données métiers béninoises de référence (écoles, filières, coûts). Le moteur IA effectue le matching sans générer les données chiffrées.
Intelligence	Google Gemini (function calling / JSON mode)	Analyse le profil collecté et retourne un JSON structuré et validé (section 5).
Export PDF	react-to-print ou html2pdf.js	Génère le PDF côté client depuis le DOM du RoadmapView, sans appel serveur.

Gestion des erreurs : la combinaison de TanStack Query et de Zod assure la gestion des timeouts réseau et le rejet des réponses non conformes au schéma, avant toute injection dans le state applicatif.

2. STRUCTURE COMPLÈTE DES DOSSIERS

Organisation en App Router, avec séparation stricte entre logique de chat, logique de restitution, et couche de données.

```
parcours-ai/
├── src/
│   ├── app/
│   │   ├── layout.tsx           # Layout racine, providers (QueryClient, fonts)
│   │   ├── page.tsx            # Landing : bouton "Démarrer mon orientation"
│   │   ├── chat/
│   │   │   ├── page.tsx        # Orchestrateur : bascule Chat <-> Roadmap
│   │   │   └── api/
│   │   │       └── chat/
│   │   │           └── route.ts # Proxy vers FastAPI (cache clé API côté serveur)
│   │
│   │   ├── components/
│   │   │   ├── chat/
│   │   │   │   ├── ChatContainer.tsx # Liste des messages + auto-scroll
│   │   │   │   ├── ChatBubble.tsx   # Bulle individuelle (user/IA)
│   │   │   │   ├── ChatInput.tsx    # Saisie + bouton envoyer (disabled pendant loading)
│   │   │   │   └── TypingLoader.tsx  # Animation "L'IA réfléchit..."
│   │   │   ├── roadmap/
│   │   │   │   ├── RoadmapView.tsx   # Conteneur des résultats (Phase 3)
│   │   │   │   ├── CareerCard.tsx    # Carte métier en accordéon (Phase 4)
│   │   │   │   ├── MatchBadge.tsx    # Badge "% de correspondance"
│   │   │   │   └── ExportPdfButton.tsx # Déclenche l'export PDF client-side
│   │   │   └── ui/                   # Composants shadcn/ui (Button, Accordion,
│   │   │       Skeleton...)
│   │
│   │   ├── store/
│   │   │   └── useChatStore.ts        # State Zustand : messages, phase, profil, résultat
│   │
│   │   ├── lib/
│   │   │   ├── schemas.ts            # Schémas Zod (validation du JSON backend)
│   │   │   └── api.ts                # Fonctions fetch typées (getNextQuestion,
│   │   │       getRoadmap)
│   │
│   │   ├── queryClient.ts            # Config TanStack Query
│   │
│   │   └── types/
│   │       └── roadmap.ts            # Types TypeScript dérivés des schémas Zod
│   │
│   └── public/
│       └── tailwind.config.ts
```

Centralisation de la validation : le regroupement des schémas Zod dans `lib/schemas.ts` constitue la source unique de vérité entre la validation runtime et le typage TypeScript, via `z.infer`.

3. FLUX DE DONNÉES ET STATE MANAGEMENT

Le store Zustand central pilote la transition entre les deux phases de l'application via une machine à états simple.

```
type ChatPhase = 'idle' | 'collecting' | 'generating' | 'completed'

idle —(clic "Démarrer")→ collecting
collecting —(échange de messages)→ collecting (boucle)
collecting —(is_final_recommendation: true)→ generating
generating —(JSON validé par Zod)→ completed
completed —(export PDF)→ completed (pas de changement d'état)
```

3.1 Forme du store Zustand

```
interface ChatStore {
  phase: 'idle' | 'collecting' | 'generating' | 'completed'
  messages: Message[]
  userProfile: Partial<UserProfile>
  roadmap: RoadmapResult | null

  startSession: () => void
  addMessage: (msg: Message) => void
  setPhase: (phase: ChatPhase) => void
  setRoadmap: (data: RoadmapResult) => void
  reset: () => void
}
```

3.2 Justification du découpage

- **Source unique de vérité** : un seul store élimine la désynchronisation entre le composant Chat et le composant Roadmap pendant la transition.
- **La phase pilote le rendu** : un enum unique `phase` détermine quel composant est monté dans `chat/page.tsx`, à la place de plusieurs booléens indépendants (`isLoading`, `isCompleted`, `isChatVisible`).
- **Persistance de session** : `phase`, `messages` et `roadmap` sont sauvegardés dans le `localStorage`. En cas d'interruption réseau, la session est restaurée à la reconnexion.

4. COMPORTEMENT UI PAR INTERACTION

Phase 1 — Lancement

Action

Clic "Démarrer mon orientation"

Réactions

- Navigation Next.js vers `/chat`.
- `startSession()` initialise le store : `phase = 'collecting'`, `messages = []`.
- TanStack Query déclenche `getFirstQuestion()` en arrière-plan.
- Apparition animée (Framer Motion, fade+slide) de la bulle IA d'accroche.

Phase 2 — Boucle conversationnelle

Action

Saisie + clic "Envoyer"

Réactions

- Mise à jour optimiste : le message utilisateur s'affiche immédiatement (`addMessage`), le champ se vide et se désactive.
- `TypingLoader` apparaît, auto-scroll vers le bas.
- Mutation TanStack Query `POST /api/chat` avec tout l'historique.
- Si succès et `is_final: false` → nouvelle bulle IA avec la question suivante.
- Si erreur réseau → message d'erreur discret + bouton "Réessayer" (pas de perte du message déjà tapé, grâce au persist).

Phase 3 — Basculement

Événement

Réponse avec

`is_final_recommendation:`
`true`

Réactions

- `setPhase('generating')` : le `ChatInput` se désactive puis disparaît (fondu, 300ms).
- Écran de chargement plein écran avec messages dynamiques rotatifs (ex. "Recherche des écoles à Cotonou...").
- Le JSON reçu passe par le schéma Zod (`RoadmapSchema.safeParse`) avant d'entrer dans le store — si invalide, retour à un état d'erreur récupérable plutôt qu'un crash.
- Si valide : `setRoadmap(data)` + `setPhase('completed')`. `<ChatContainer>` est démonté, `<RoadmapView>` est monté avec une transition Framer Motion.

Phase 4 — Consultation et export

Action

Clic sur une CareerCard

Réactions

- Accordéon shadcn/ui : la carte se déplie (hauteur animée).
- Affichage des écoles recommandées, coûts en FCFA, compétences à acquérir.

Action

Clic "Télécharger en PDF"

Réactions

- Capture du DOM de `RoadmapView` via `react-to-print` ou `html2pdf.js` — génération 100% client, aucun appel serveur.
- Le navigateur propose l'enregistrement de `Mon_Parcours_ParcoursAI.pdf`.

5. SCHÉMA JSON STRICT (CONTRAT BACKEND)

Le prompt système envoyé à Gemini doit forcer une réponse conforme à ce schéma exact. Deux formes possibles selon l'état de la collecte :

5.1 Réponse intermédiaire (collecte en cours)

```
{
  "is_final_recommendation": false,
  "question": "Quel est le budget annuel approximatif de ta famille pour tes études (en FCFA) ?",
  "field_expected": "budget_fcfa"
}
```

5.2 Réponse finale (roadmap générée)

```
{
  "is_final_recommendation": true,
  "user_profile_summary": {
    "serie_bac": "Tle C",
    "ville": "Cotonou",
    "budget_fcfa": 800000,
    "interets": ["informatique", "mathématiques"]
  },
  "recommendations": [
    {
      "id": "dev-web",
      "metier": "Développeur Web",
      "match_percentage": 92,
      "description_courte": "Conception et développement d'applications et de sites web.",
      "competences_a_acquerir": ["JavaScript", "Bases de données", "Algorithmique"],
      "etablisements": [
        {
          "nom": "IFRI (UAC)",
          "ville": "Abomey-Calavi",
          "frais_annuels_fcfa": 150000,
          "duree_ans": 3
        },
        {
          "nom": "Epitech Bénin",
          "ville": "Cotonou",
          "frais_annuels_fcfa": 1200000,
          "duree_ans": 5
        }
      ]
    }
  ]
}
```

Validation Zod correspondante (extrait) : chaque champ numérique (frais, pourcentage) doit être typé strictement (`z.number().int().positive()`) pour rejeter toute hallucination de type string ou valeur aberrante avant l'affichage.

6. SPÉCIFICATIONS DE PERFORMANCE (FRONTEND)

- **Streaming des questions** : le rendu par streaming de tokens affiche la question mot par mot, réduisant la latence perçue sur mobile.
- **Chargement dynamique** : `RoadmapView` et la librairie d'export PDF sont chargées via `next/dynamic`, en dehors du bundle initial du chat.
- **Images des établissements** : format WebP via `next/image`, avec placeholder flou, pour un chargement optimisé sur réseau mobile.
- **Verrouillage du bouton d'envoi** : le bouton "Envoyer" est désactivé pendant une requête en cours, empêchant les doubles soumissions.

7. ARCHITECTURE BACKEND — STRUCTURE FASTAPI

Le backend a deux responsabilités : sécuriser la clé API Gemini (jamais exposée au client) et orchestrer le contrat JSON strict entre l'IA et le frontend.

```
backend/
├── app/
│   ├── main.py                # Point d'entrée FastAPI, config CORS
│   ├── routers/
│   │   ├── chat.py           # POST /chat -> question suivante ou roadmap finale
│   │   └── health.py         # GET /health -> ping pour monitoring
│   ├── services/
│   │   ├── gemini_service.py  # Appel à l'API Gemini, gestion du prompt système
│   │   ├── matching_service.py # Croise le profil utilisateur avec les fiches métiers
│   │   └── supabase_service.py # Client Supabase (lecture des données de référence)
│   ├── schemas/
│   │   ├── chat_schema.py     # Pydantic : ChatRequest, ChatResponse
│   │   └── roadmap_schema.py  # Pydantic : RoadmapResult, CareerRecommendation
│   ├── data/
│   │   └── fiches_metiers.json # Fallback local des fiches métiers (voir section 9)
│   └── core/
│       ├── config.py          # Variables d'env (clé Gemini, URL Supabase)
│       └── prompts.py         # Templates du prompt système (voir section 10)
├── requirements.txt
└── .env
```

Séparation des responsabilités : `gemini_service.py` extrait les critères structurés du profil (série, budget, intérêts) à partir du langage naturel. `matching_service.py`, code Python déterministe, calcule le `match_percentage` et sélectionne les établissements dans Supabase. Le calcul des frais de scolarité et le choix des établissements ne dépendent à aucun moment de l'inférence du modèle.

7.1 Endpoint principal

POST /chat

Body: { messages: Message[], user_profile: Partial<UserProfile> }

Logique interne (chat.py) :

1. Reçoit l'historique complet + profil partiel accumulé
2. Appelle `gemini_service.extract_and_decide(messages, profile)`
3. Si profil incomplet -> Gemini renvoie la prochaine question
=> `ChatResponse(is_final_recommendation=False, question=...)`
4. Si profil complet -> `matching_service.compute_recommendations(profile)`
=> interroge Supabase, calcule les scores, construit le JSON final
=> `ChatResponse(is_final_recommendation=True, recommendations=[...])`
5. Validation Pydantic automatique avant retour (garantit le contrat JSON)

8. MODÈLE DE DONNÉES SUPABASE

Supabase (Postgres) stocke les données de référence béninoises — la vérité terrain que l'IA ne doit pas halluciner.

TABLE	COLONNES CLÉS	RÔLE
metiers	id, nom, description, secteur, competences_requises (jsonb)	Référentiel des métiers proposés par l'application.
etablisements	id, nom, ville, type (public/privé), site_web	Écoles et universités béninoises (UAC, IFRI, Epitech, etc.).
formations	id, etablissement_id (FK), metier_id (FK), frais_annuels_fcfa, duree_ans, niveau_requis	Table de liaison : quelle formation, dans quel établissement, prépare à quel métier, pour quel coût.
sessions_utilisateur	id, session_uuid, profil_collecte (jsonb), created_at	Historique anonymisé des profils collectés — utile pour les statistiques post-hackathon (aucune donnée personnelle identifiable stockée).

8.1 Requête de matching type

```
SELECT m.nom AS metier, f.frais_annuels_fcfa, f.duree_ans,
       e.nom AS etablissement, e.ville
FROM formations f
JOIN metiers m ON m.id = f.metier_id
JOIN etablissements e ON e.id = f.etablissement_id
WHERE f.frais_annuels_fcfa <= :budget_fcfa
      AND e.ville = ANY(:villes_accessibles)
ORDER BY m.match_score DESC
LIMIT 3;
```

Format JSONB : le champ `competences_requises` est stocké en JSONB, permettant l'évolution de la structure sans migration de schéma.

9. FICHES MÉTIERS BÉNINOISES (DONNÉES DE RÉFÉRENCE)

Ces fiches constituent la base de connaissance contextualisée qui différencie PARCOURS-AI d'un simple chatbot générique.

```
{
  "id": "dev-web",
  "nom": "Développeur Web",
  "secteur": "Numérique",
  "description": "Conception et développement d'applications et de sites web.",
  "competences_requises": ["JavaScript", "Bases de données", "Algorithmique"],
  "series_bac_recommandees": ["C", "D", "TI"],
  "debouches_locaux": ["Startups tech de Cotonou", "Agences web", "Freelance"],
  "formations_associees": [
    { "etablissement": "IFRI (UAC)", "ville": "Abomey-Calavi" },
    { "etablissement": "Epitech Bénin", "ville": "Cotonou" }
  ]
}
```

- **Source de vérité double** : Supabase constitue la source en production ; `data/fiches_metiers.json` sert de fallback local en cas d'indisponibilité de la connexion Supabase.
- **Extensibilité** : chaque fiche admet des champs optionnels, permettant un enrichissement (déroulés, témoignages) sans rupture du contrat JSON existant.
- **Alimentation initiale** : la base de référence comprend une vingtaine de fiches couvrant les filières suivantes : numérique, santé, droit, gestion, BTP.

10. INTERACTION AVEC L'API GEMINI

Le prompt système doit forcer Gemini à jouer deux rôles distincts : extracteur de critères structurés, puis rédacteur de questions naturelles — jamais l'inverse (jamais de calcul de prix ou de matching par l'IA elle-même).

10.1 Structure du prompt système

Rôle : assistant d'orientation de PARCOURS-AI au Bénin.

RÈGLES STRICTES :

1. Poser UNE seule question à la fois, en français, sur un ton adapté à un adolescent.
2. Ne jamais indiquer de nom d'établissement, de métier ou de frais : ces données proviennent exclusivement de la base Supabase.
3. La sortie est exclusivement un objet JSON conforme au schéma fourni (voir section 5). Aucun texte hors JSON, aucun markdown, aucune explication.
4. Champs à collecter avant `is_final_recommendation=true` : `serie_bac`, `resultats_scolaires`, `appetences`, `budget_fcfa`, `ville`.
5. Dès que les 5 champs sont renseignés, retourner `is_final_recommendation=true` avec `user_profile_summary` rempli (sans la liste `recommendations`, calculée par le backend).

Répartition des responsabilités : en phase finale, l'API Gemini retourne exclusivement `user_profile_summary`, à l'exclusion de la liste `recommendations`. Cette liste est construite par `matching_service.py` (section 7) à partir des données Supabase. Le calcul des frais de scolarité et la sélection des établissements sont ainsi indépendants de l'inférence du modèle.

10.2 Configuration de l'appel

```
model = "gemini-2.0-flash"
generation_config = {
    "response_mime_type": "application/json",
    "response_schema": ChatResponseSchema # schéma Pydantic converti
}
```

11. SÉCURITÉ ET ROBUSTESSE BACKEND

- **Clé API Gemini** : stockée uniquement côté serveur (variable d'environnement), jamais transmise au frontend — tout appel passe par `/api/chat` côté Next.js puis FastAPI.
- **Validation Pydantic systématique** : toute sortie de `gemini_service` est validée contre le schéma avant d'atteindre le frontend ; en cas d'échec, retry automatique avec une reformulation du prompt (1 tentative max pour ne pas ralentir l'UX).
- **Rate limiting** : les appels sont limités par session (20 messages maximum), contenant les coûts d'appel à l'API.
- **Fallback local** : en cas d'indisponibilité de Supabase, bascule automatique sur `fiches_metiers.json`, garantissant la continuité du service.
- **CORS restreint** : uniquement le domaine du frontend déployé (Vercel) autorisé à appeler l'API FastAPI.

12. CODE DES ENDPOINTS FASTAPI

Implémentation des fichiers clés définis en section 7.

12.1 schemas/chat_schema.py

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

class Message(BaseModel):
    role: Literal["user", "assistant"]
    content: str

class UserProfile(BaseModel):
    serie_bac: Optional[str] = None
    resultats_scolaires: Optional[str] = None
    appetences: Optional[list[str]] = None
    budget_fcfa: Optional[int] = None
    ville: Optional[str] = None

class ChatRequest(BaseModel):
    messages: list[Message]
    user_profile: UserProfile = Field(default_factory=UserProfile)

class Etablissement(BaseModel):
    nom: str
    ville: str
    frais_annuels_fcfa: int
    duree_ans: int

class CareerRecommendation(BaseModel):
    id: str
    metier: str
    match_percentage: int = Field(ge=0, le=100)
    description_courte: str
    competences_a_acquerir: list[str]
    etablissements: list[Etablissement]

class ChatResponse(BaseModel):
    is_final_recommendation: bool
    question: Optional[str] = None
    field_expected: Optional[str] = None
    user_profile_summary: Optional[UserProfile] = None
    recommendations: Optional[list[CareerRecommendation]] = None
```

12.2 routers/chat.py

```
from fastapi import APIRouter, HTTPException
from app.schemas.chat_schema import ChatRequest, ChatResponse
from app.services import gemini_service, matching_service

router = APIRouter()

@router.post("/chat", response_model=ChatResponse)
async def chat_endpoint(payload: ChatRequest):
    try:
        extraction = await gemini_service.extract_and_decide(
            messages=payload.messages,
            current_profile=payload.user_profile,
        )
    except gemini_service.GeminiValidationError:
        # Une seule reformulation automatique avant d'échouer proprement
        extraction = await gemini_service.extract_and_decide(
            messages=payload.messages,
            current_profile=payload.user_profile,
            retry=True,
        )
    except Exception:
        raise HTTPException(status_code=502, detail="Service IA indisponible")

    if not extraction.is_complete:
        return ChatResponse(
            is_final_recommendation=False,
            question=extraction.next_question,
            field_expected=extraction.field_expected,
        )

    recommendations = matching_service.compute_recommendations(
        profile=extraction.profile
    )
    return ChatResponse(
        is_final_recommendation=True,
        user_profile_summary=extraction.profile,
        recommendations=recommendations,
    )
```

12.3 services/matching_service.py

```
from app.services import supabase_service
from app.schemas.chat_schema import UserProfile, CareerRecommendation

def compute_recommendations(profile: UserProfile) -> list[CareerRecommendation]:
    '''Calcul deterministe : l'IA ne touche jamais aux chiffres FCFA.'''
    candidats = supabase_service.get_formations_compatibles(
        budget_max=profile.budget_fcfa,
        ville=profile.ville,
    )

    scores = [
        (formation, _score_affinite(formation, profile.appetences))
        for formation in candidats
    ]
    scores.sort(key=lambda x: x[1], reverse=True)

    return [
        _to_recommendation(formation, score)
        for formation, score in scores[:3]
    ]

def _score_affinite(formation, appetences: list[str]) -> int:
    mots_cles = set(formation["mots_cles"])
    intersection = mots_cles & set(appetences or [])
    return round(100 * len(intersection) / max(len(mots_cles), 1))

def _to_recommendation(formation, score) -> CareerRecommendation:
    return CareerRecommendation(
        id=formation["metier_id"],
        metier=formation["metier_nom"],
        match_percentage=score,
        description_courte=formation["description"],
        competences_a_acquerir=formation["competences"],
        etablissements=formation["etablissements"],
    )
```

12.4 services/gemini_service.py (extrait)

```
import google.generativeai as genai
from app.core.config import GEMINI_API_KEY
from app.core.prompts import SYSTEM_PROMPT
from app.schemas.chat_schema import ChatResponse

genai.configure(api_key=GEMINI_API_KEY)

model = genai.GenerativeModel(
    "gemini-2.0-flash",
    system_instruction=SYSTEM_PROMPT,
    generation_config={"response_mime_type": "application/json"},
)

class GeminiValidationError(Exception):
    pass

async def extract_and_decide(messages, current_profile, retry: bool = False):
    history = _format_history(messages)
    response = model.generate_content(history)

    try:
        return _parse_and_validate(response.text, current_profile)
    except ValueError as exc:
        if retry:
            raise GeminiValidationError(str(exc))
        raise
```

État d'implémentation : le code constitue un squelette fonctionnel. Les fonctions `_format_history`, `_parse_and_validate` et `supabase_service.get_formation_compatibles` restent à implémenter selon le schéma exact de la base Supabase.